



Google Summer of Code



GSoC 2026: Generalizing Neural-LAM to Probabilistic Forecasting Models

Aswani Kumar Sahoo

Project Idea #3 @ MLLAM | 350 hours | Large

Field	Details
Name	Aswani Kumar Sahoo
Email	aswanisahoo227@gmail.com
GitHub	github.com/AswaniSahoo
University	National Institute of Technology Rourkela, Odisha, India (B.Tech, 2027)
Timezone	IST (UTC+5:30)
Availability	35 hours/week, no other summer commitments (exams finish early April, before Community Bonding)
Computational Resources:	Local NVIDIA RTX 3050 (4GB VRAM) for PoC prototyping and unit tests. Kaggle P100 (16GB, 30 hr/week) and Google Colab T4 for training runs. For Graph-EFM on MEPS: reduced ensemble sizes ($S=2-4$) with gradient accumulation; full evaluation ($S=10+$) via sequential trajectory sampling to avoid OOM. Exact memory/batch-size constraints will be profiled during community bonding Week 2. If larger-scale training is needed (e.g., full paper reproduction), I will coordinate with mentors on compute access during Phase 2.
Primary Preference	MLLAM is my first-choice organization
Mentors	Joel Oskarsson (@joeloskarsson), Hauke Schulz (@observingClouds), Leif Denby (@leifdenby)

1. Abstract

Neural-LAM's current `ARModel` (772 lines) mixes five responsibilities into a single class: Lightning lifecycle management, autoregressive rollout, single-step prediction, loss computation, and metric aggregation/plotting. This monolithic design prevents the integration of probabilistic forecasting models like Graph-EFM (Oskarsson et al., 2024)

and Diffusion-LAM (Oskarsson et al., 2025), which require fundamentally different training loops (ELBO vs MSE) and prediction modes (ensemble sampling vs point estimates).

This project will:

1. **Refactor the model class hierarchy** (Issue #49) into composable `ForecasterModule` → `Forecaster` → `StepPredictor` components
2. **Merge the Graph-EFM model** (Issue #62) from the `prob_model_lam` branch into the new hierarchy as the first probabilistic model
3. **Add ensemble evaluation infrastructure** – CRPS, spread-skill metrics, and ensemble visualization

I bring direct experience with this exact problem domain: I implemented a DDPM-based `ThermalizerLayer` for weather forecasting at Open Climate Fix (3 merged PRs in `graph_weather`), built a Weather Transformer from scratch on ERA5/WeatherBench2 data, and have performed a method-level classification of every function in `ar_model.py` mapped to the proposed #49 hierarchy.

2. Project Understanding

2.1 The Problem: Why ARModel is a Monolith

After reading `ar_model.py` on `main` and `graph_efm.py` on `prob_model_lam`, I performed a method-level classification of all 18 methods in `ARModel`. Each category below represents one of the proposed new classes from Issue #49 ; i.e., where that method should live after refactoring. The fact that all of them currently sit in a single `ARModel` class is the core problem:

Category	Methods	Problem
[FORECASTER_MODULE] (<i><code>pl.LightningModule</code> : thin training wrapper</i>)	<code>__init__</code> , <code>configure_optimizers</code> , <code>plot_examples</code> , <code>on_load_checkpoint</code> , <code>all_gather_cat</code> , <code>_create_dataarray_from_tensor</code>	Lightning lifecycle, logging, device handling, plotting dispatch
[AR_FORECASTER] (<i><code>nn.Module</code> : autoregressive loop</i>)	<code>unroll_prediction</code> , <code>common_step</code> , <code>interior_mask_bool</code>	AR rollout with boundary masking : shared between deterministic and probabilistic models
[STEP_PREDICTOR] (<i><code>nn.Module</code> : single-step prediction</i>)	<code>predict_step</code> (abstract), <code>expand_to_batch</code>	Single-step prediction interface : overridden by concrete models (GraphLAM, HiLAM, etc.)

Category	Methods	Problem
[LOSS] (should be overridable in ForecasterModule)	training_step (partial), validation_step (partial), test_step (partial)	Loss computation : currently hardcoded to MSE, needs to support ELBO for probabilistic models
[METRICS] (composable in ForecasterModule)	create_metric_log_dict, aggregate_and_plot_metrics, on_validation_epoch_end, on_test_epoch_end	Metric aggregation and logging : needs extension for ensemble metrics (CRPS, spread-skill)

The three most tangled methods are `training_step`, `validation_step`, and `test_step` : each simultaneously handles loss computation, metric logging, and Lightning lifecycle concerns, making them the hardest to cleanly separate.

2.2 Why This Blocks Probabilistic Models

`GraphEFM` on `prob_model_lam` (1129 lines) demonstrates the coupling problem:

- **training_step is completely overridden** : replaces `common_step` → `unroll_prediction` → MSE loss with a per-step ELBO loop computing `likelihood +  $\beta$  · KL divergence + optional CRPS`
- **predict_step has two modes** : prior sampling (inference, via `grid_prev_embedder`) vs variational inference (training, via `grid_current_embedder` which sees the target state through `embedd_current`)
- **GraphEFM bypasses BaseGraphModel entirely** : inherits directly from `ARModel`, duplicating graph loading and embedding setup
- **Batch format difference**: `prob_model_lam` uses 3-element batches vs main's 4-element batches (minor, easily unified by adding `batch_times`)
- **Hardcoded wandb.log()** : main has been refactored to use Lightning's generic logger
- **constants module references** : main uses the `datastore` API instead
- **plot_examples signature differs** : `GraphEFM` omits the `split` parameter, causing an API incompatibility
- **on_validation_epoch_end ordering** : `GraphEFM` logs spread-skill *before* `super()` (which clears lists), but `on_test_epoch_end` calls `super()` *first*, an asymmetry that needs harmonizing

These conflicts mean Graph-EFM cannot be merged without the hierarchy refactoring first, exactly what Joel stated in Issue #62: *"I think it is best to do this merge after #49 is done, so the new model can fit into the new class hierarchy."*

2.3 The Dual Training Path Problem

The fundamental architectural challenge is that deterministic and probabilistic models have **different training loops**:

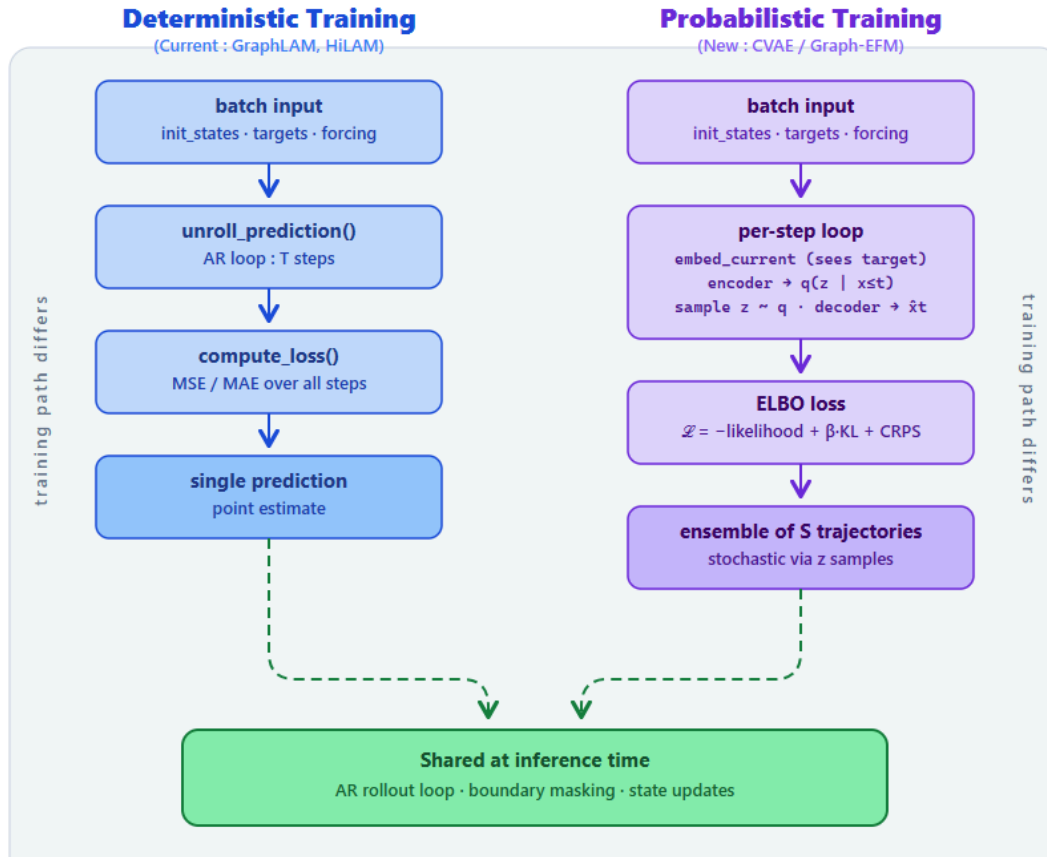


Figure 1: Deterministic vs Probabilistic training paths. Both share the same AR rollout loop, boundary masking, and state updates at inference time.

Yet they share the same AR rollout logic **at inference time**:

```
# This inference-time loop is identical for both:
for t in range(pred_steps):
    pred_state = step_predictor(prev_state, prev_prev_state, forcing)
    new_state = boundary_mask * true_state + interior_mask * pred_state
    prev_prev_state = prev_state
    prev_state = new_state
```

During training, the CVAE additionally needs target access (for the encoder's variational posterior), but the boundary masking and state update logic remains shared. This is the core modularity argument: the **AR rollout** should be shared, while the **step prediction** and **loss computation** should be pluggable.

3. Proposed Architecture

Following Joel's design from Issue #49, with modifications informed by my analysis of GraphEFM:

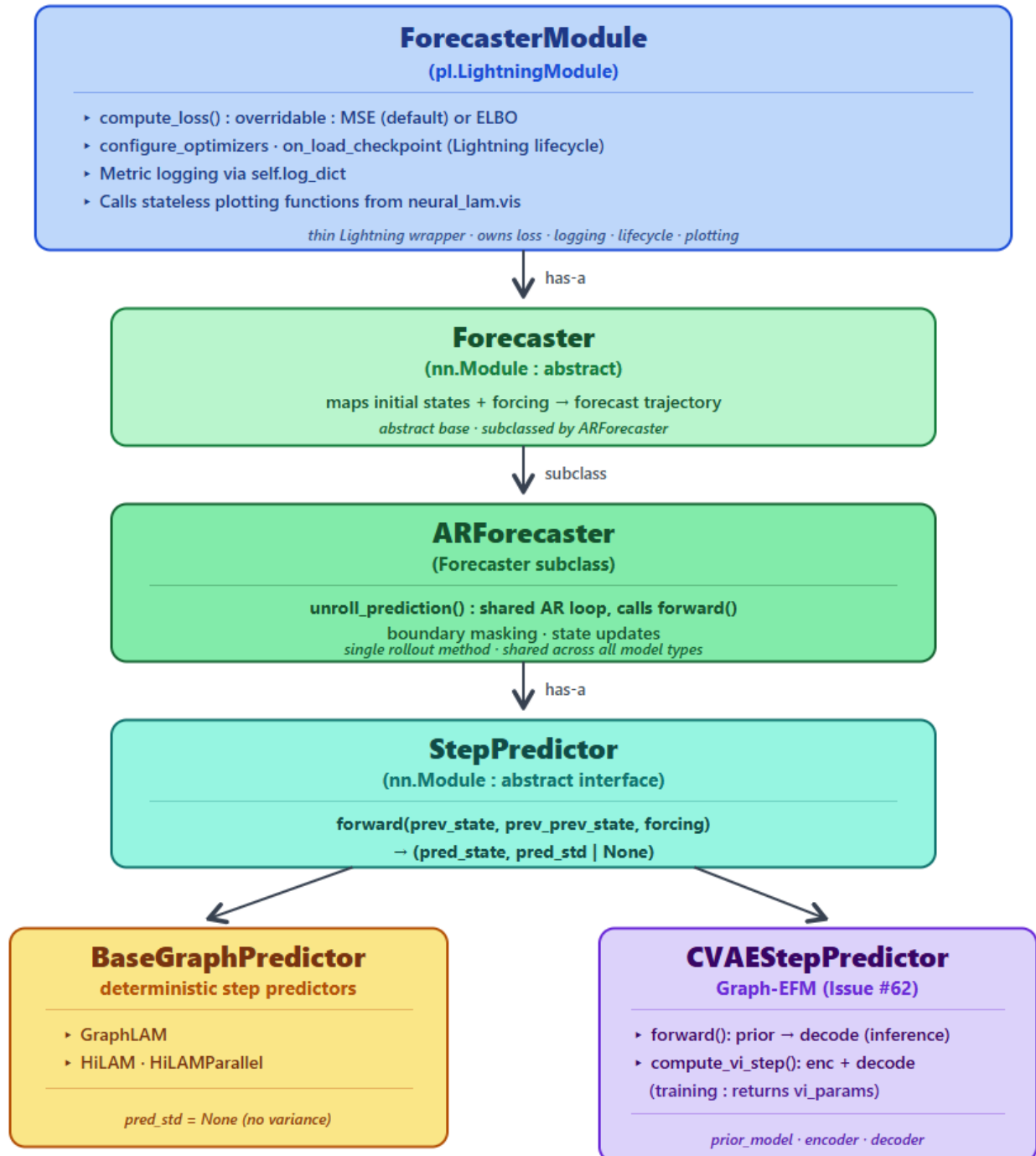


Figure 2: Proposed class hierarchy. `ForecasterModule` (`pl.LightningModule`) owns loss, logging, and lifecycle. `ARForecaster` owns the rollout loop. `StepPredictor` is pluggable : deterministic or probabilistic.

```

ForecasterModule(pl.LightningModule)
├─ compute_loss() - overridable: MSE (default) or ELBO
├─ configure_optimizers, on_load_checkpoint (Lightning lifecycle)
├─ Metric logging via self.log_dict
├─ Calls stateless plotting functions from neural_lam.vis
└─ has-a → Forecaster (nn.Module)
    └─ ARForecaster
        ├─ unroll_prediction (shared AR loop, calls forward())
        └─ has-a → StepPredictor (nn.Module)
            ├─ BaseGraphPredictor → GraphLAM, HiLAM, ..
            └─ CVAEStepPredictor (Graph-EFM)
                ├─ forward() - inference (prior → decode)
                └─ compute_vi_step() - training (encoder +
decode)

```

3.1 Component Responsibilities

ForecasterModule(pl.LightningModule) : What remains from ARModel:

- `configure_optimizers, on_load_checkpoint` (Lightning lifecycle)
- Batch unpacking and device handling
- `compute_loss(prediction, target, pred_std, vi_params, ...)`: overridable method. Default computes MSE (ignores `vi_params`).
- Calls stateless plotting functions from `neural_lam.vis`
- Metric logging via Lightning's `self.log_dict`

For probabilistic models, a **ProbabilisticForecasterModule** subclass overrides `compute_loss` to minimize the negative ELBO: $-\text{likelihood} + \beta \cdot \text{KL}$, plus optional CRPS regularization. It also adds ensemble metric logging. This is the "unmentioned class" that bridges the hierarchy with probabilistic training.

Forecaster(nn.Module): Generic forecast generation:

- Abstract: maps initial states + forcing → full forecast trajectory
- **ARForecaster** subclass: the autoregressive loop from `unroll_prediction` + boundary masking. During inference, calls `step_predictor.forward()` at each step. Ensemble generation sits **outside** **ARForecaster** (see section - 3.2 for tensor shape contract per [Issue #335](#)).

StepPredictor(nn.Module): Single-step prediction:

- `forward(prev_state, prev_prev_state, forcing) → (pred_state, pred_std | None)`
- Returns `None` for `pred_std` when not predicting variance (deterministic models)
- For models that explicitly predict output variance (e.g., Gaussian likelihood), returns `pred_std`. For latent variable models like CVAE, `pred_std` may be `None` since uncertainty is captured through `z` samples and the decoder outputs the mean prediction only. In either case, `pred_std` is used only for loss computation, **not** for AR state updates (Joel confirmed sampling with `pred_std` is "a very bad idea in practice")

- Probabilistic subclasses additionally implement a training method (e.g. `compute_vi_step`) returning `(pred_state, pred_std | None, vi_params)`
- Existing `BaseGraphModel` → `BaseGraphPredictor` (renamed per Leif's suggestion)
- New `CVAEStepPredictor`: wraps Graph-EFM's dual-path architecture. Following Joel's guidance (Mar 28) that `forward()` is best reserved for inference (sampling one step ahead from the prior), the CVAE step predictor separates inference and training into distinct methods:

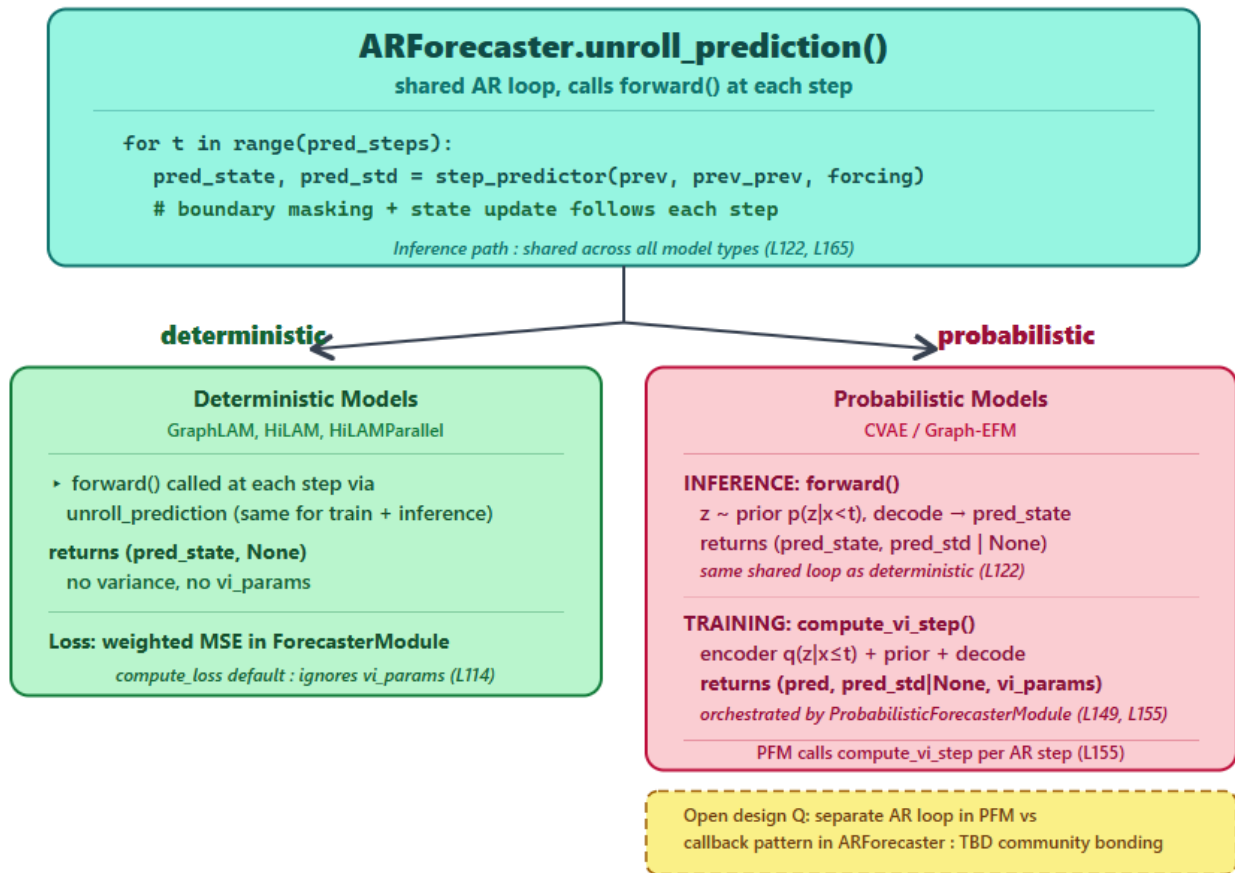
```
class CVAEStepPredictor(StepPredictor):
    def forward(self, prev_state, prev_prev_state, forcing):
        """Inference: sample z from prior, decode one step."""
        z, _ = self.prior(prev_state, ...)          #  $p(z|x_{<t})$ 
        return self.decoder(z, prev_state, forcing), None

    def compute_vi_step(self, prev_state, prev_prev_state, forcing,
                        current_state):
        """Training: compute encoder quantities for ELBO."""
        z_prior, prior_params = self.prior(prev_state, ...)
        z_enc, enc_params = self.encoder(current_state, ...)  #  $q(z|x_{\leq t})$ 
        vi_params = {"enc": enc_params, "prior": prior_params}
        pred = self.decoder(z_enc, prev_state, forcing)
        return pred, None, vi_params
```

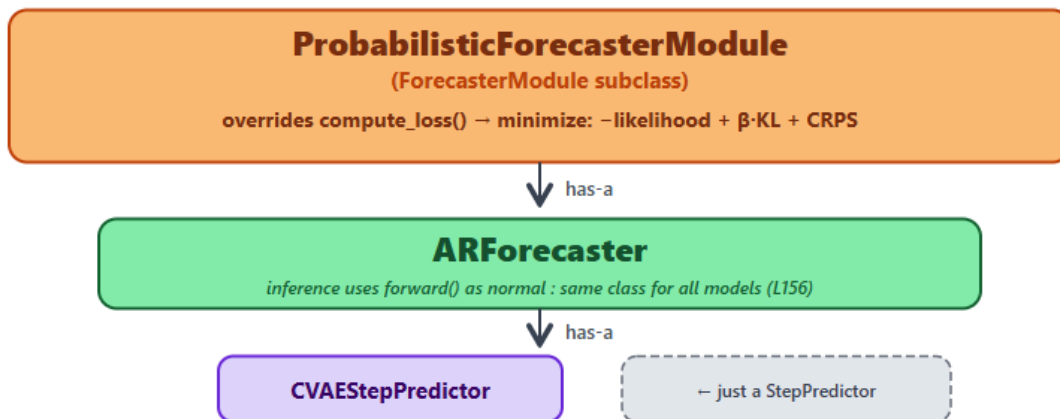
During inference, `forward()` samples z from the prior - a clean, standard PyTorch forward pass. During VI training, `ProbabilisticForecasterModule` calls `compute_vi_step()` instead, which additionally runs the encoder on the target state to produce the distribution parameters needed for KL divergence. Deterministic `StepPredictor` subclasses have no `compute_vi_step()` and always use `forward()`. The exact naming and interface of this training method will be finalized with mentors during Phase 0.

How they compose for probabilistic models:

```
ProbabilisticForecasterModule(ForecasterModule)
├─ overrides compute_loss for ELBO
├─ orchestrates training rollout (calls compute_vi_step per step)
└─ has-a → ARForecaster (inference uses forward() as normal)
    └─ has-a → CVAEStepPredictor(StepPredictor)
```



How they compose for probabilistic models:



CVAEStepPredictor **never** interacts with ForecasterModule directly ; it is composed inside ARForecaster like any other StepPredictor. The ELBO loss computation stays in ProbabilisticForecasterModule, keeping the hierarchy clean.

3.2 Key Design Decisions

Where does loss go? Joel noted he was "not particularly happy to send the target tensor further down the hierarchy." My proposed solution:

`ProbabilisticForecasterModule` owns `compute_loss(prediction, target, pred_std, vi_params, ...)` as an overridable method. The default (`ForecasterModule`) computes weighted MSE (ignores `vi_params`). For ELBO training, `ProbabilisticForecasterModule` uses `vi_params` (the per-step encoder/prior μ, σ accumulated by `ARForecaster`) to minimize $-\text{likelihood} + \beta \cdot \text{KL}(q||p) + \text{optional CRPS}$. Note: for CVAE models, the target *does* need to reach the step predictor (for the encoder's variational posterior) and this is inherent to variational inference and cannot be avoided. However, the **loss computation and metric logging** remain exclusively in `ForecasterModule`, keeping the separation clean.

How does the training path work? During inference, `ARForecaster` calls `step_predictor.forward()` at each step: this path is shared across all model types. During training for probabilistic models, `ProbabilisticForecasterModule` orchestrates a separate path that calls the step predictor's training method (e.g. `compute_vi_step()`) to obtain both predictions and VI parameters at each AR step. This keeps `forward()` clean for inference while allowing probabilistic models to compute encoder quantities. The exact mechanism, whether this is a separate AR loop in `ProbabilisticForecasterModule`, or a callback pattern within `ARForecaster`, is an open design question for community bonding. Joel confirmed this direction (Mar 28): *"forward is best used just as sampling a prediction one time step ahead."*

Ensemble tensor shapes follow the contract proposed in Issue #335 (Joel, Mar 8): `StepPredictor` always maps $(B, N, F) \rightarrow (B, N, F)$ (single-trajectory, either deterministic or stochastic), `ARForecaster` returns (B, T, N, F) , and ensemble generation folds S into the batch dimension $(S*B, T, N, F)$ outside `ARForecaster`. Latent variables z_t are sampled independently per batch item (Joel, Mar 15), making the fold transparent to the CVAE internals.

Spatial dimensions: Keep the current convention (flatten in `DataLoader`). As Joel noted: *"once we start moving to more refined boundary setups, there will no longer be a 2d grid representation."* `StepPredictors` that need 2D (CNN/ViT) can reshape internally.

Relationship to PR #208: I reviewed PR #208 by @Sir-Sloth-The-Lazy, which implements the initial deterministic hierarchy refactoring. Joel's review (Mar 1) confirmed several key design decisions my proposal incorporates:

- **Composition pattern:** `ForecasterModule` takes a `Forecaster` as constructor argument (not internal instantiation)
- **`pred_std | None` interface:** `StepPredictor` returns `None` when not predicting variance; `ForecasterModule` decides loss weighting

- **Explicit parameters:** No `args` namespace passed to constructors; use keyword arguments
- **Boundary ownership:** StepPredictor does not track boundary masks; that's the Forecaster's concern

My project builds **on top of PR #208**, not as a re-implementation. Phase 1 assumes #208 is merged (or close to merging) and focuses on extending the hierarchy for probabilistic support, work that #208 does not address. During community bonding, I would contribute to #208's review process to help get it merged.

4. Implementation Plan (350 hours)

Phase 0: Community Bonding (May 1–24, ~50 hours)

Week	Task	Deliverable
1	Finalize design with mentors; review PR #208's approach and any feedback. Align on naming conventions and exact hierarchy.	Design document agreed upon
2	Study <code>prob_model_lam</code> branch end-to-end; document all merge conflicts. Reproduce MEPS training with existing deterministic models to establish baseline.	Conflict resolution plan + baseline training log
3	PoC: Minimal <code>ForecasterModule</code> + <code>ARForecaster</code> skeleton that wraps existing <code>GraphLAM</code> and passes existing tests, building on any merged plotting extractions (e.g. PR #209).	Working PoC branch, all existing tests passing

Phase 1: Extending the Hierarchy for Probabilistic Support (Weeks 1–6, ~175 hours)

This phase builds on PR #208's deterministic hierarchy. PR #208 is in near-final review (Joel self-assigned, added to v0.7.0 milestone, described the PR as "really good" on Mar 22). Phase 1 assumes #208 is merged and focuses on **extending** the hierarchy to support probabilistic models and the part that #208 does not address. All probabilistic extensions (KL loss integration, ensemble handling) will be submitted as **separate follow-up PRs** on top of the merged deterministic base, per Joel's guidance on scope boundaries.

Week	Task	Deliverable	Acceptance Criteria
Week 1	Verify #208's merged hierarchy works end-to-end; run full test suite on MEPS. Identify any gaps or adjustments needed for probabilistic extension.	Baseline validation log + gap analysis	All existing tests pass on merged hierarchy
Week 2	Add training-time interface for probabilistic step predictors (separate from <code>forward()</code> , per Joel's guidance). Add <code>ProbabilisticForecasterModule</code> subclass with overridable <code>compute_loss</code> that orchestrates the training path.	Extended <code>StepPredictor</code> or interface, new <code>ProbabilisticForecasterModule</code>	Deterministic models still pass all tests (no regression)
Week 3	Port <code>PropagationNet</code> as a standalone PR (can benefit existing deterministic models per Joel's guidance). Separately port latent encoder/decoder hierarchy from <code>prob_model_lam</code> . Replace <code>constants</code> references with <code>datastore</code> API. Fix <code>border_mask</code> → <code>boundary_mask</code> .	New files in <code>neural_lam/models/</code>	Unit tests pass for each ported component
Week 4	Implement <code>CVAEStepPredictor</code> : wrap Graph-EFM's dual-embedder architecture with separate <code>forward()</code> (inference) and <code>compute_vi_step()</code> (training) methods.	<code>CVAEStepPredictor</code> class	Prior sampling produces valid shaped outputs
Week 5	Implement ELBO training path in <code>ProbabilisticForecasterModule</code> : minimize $-\text{likelihood} + \beta \cdot \text{KL} + \text{optional CRPS}$. Add <code>sample_trajectories</code> for ensemble generation.	ELBO training working	Loss decreases, KL annealing functions
Week 6	Midterm integration: train Graph-EFM through new hierarchy on MEPS. Documentation, cleanup. Midterm evaluation.	PR ready for review	Graph-EFM trains through hierarchy, deterministic models unaffected

Midterm Deliverable: Graph-EFM is trainable through the extended hierarchy. Deterministic models still work unchanged. Inference uses `forward()`, training uses the separate `compute_vi_step()` path.

Phase 2: Ensemble Evaluation & Polish (Weeks 7–12, ~175 hours)

Week	Task	Deliverable	Acceptance Criteria
Week 7	Build on PR #226's ensemble metrics (fair/unfair/almost-fair CRPS, <code>spread_squared</code>). Integrate with <code>ProbabilisticForecasterModule</code> . Following Joel's guidance: marginal CRPS is the primary target ; joint trajectory statistics are an open research problem and explicitly out of scope.	Extended <code>metrics.py</code> integrated with new hierarchy	CRPS outputs numerically match PR #226's implementation on identical inputs; all three estimator modes (fair/unfair/almost-fair) pass unit tests
Week 8	Port ensemble visualization: <code>plot_ensemble_prediction</code> , <code>plot_latent_samples</code> from <code>vis.py</code> . Align <code>plot_examples</code> signature across deterministic and probabilistic modes.	Extended <code>vis.py</code>	Ensemble plots generated correctly
Week 9	End-to-end integration testing: train Graph-EFM on MEPS data, generate ensemble forecasts, evaluate CRPS and spread-skill ratio (Issue #404) metrics.	Training artifacts + validation results	Full pipeline working
Week 10	Robustness testing: multi-GPU validation, edge cases, performance profiling. Address any issues found during integration.	Test reports + fixes	Stable across configurations
Week 11	Documentation: update README, add usage examples for probabilistic models, document the extension pattern for adding new probabilistic models. Code freeze.	Complete documentation	Clear docs for contributors
Week 12	Final reviewer feedback, cleanup, prepare submission. Buffer for any remaining issues..	Final PR(s) ready	Graph-EFM trains and evaluates through new hierarchy

Final Deliverable: Graph-EFM is runnable on `main` through the new hierarchy. Ensemble forecasts can be generated, evaluated (CRPS, spread-skill ratio per Issue #404), and visualized.

Stretch Goal: Diffusion-LAM Scaffold

If Phase 2 completes ahead of schedule, I will implement a `DiffusionStepPredictor` scaffold that demonstrates how the iterative denoising process fits under the same `StepPredictor` interface. This connects to ErikLarssonDev's work in `real-prob-lam` and positions the architecture for future Diffusion-LAM integration. My DDPM ThermalizerLayer at OCF(Open Climate Fix) provides direct implementation experience for this.

5. Why I Am the Right Person

5.1 Production Weather ML Contributions – 3 Merged PRs at Open Climate Fix

3 merged PRs at Open Climate Fix's `graph_weather`:

PR	What I Built	Relevance to Project 3
#166 – ThermalizerLayer (455 additions)	DDPM diffusion layer with UNet architecture, cosine beta scheduling, positional encoding conditioning, unit + integration tests	Directly relevant to Diffusion-LAM. I solved the exact conditioning-path separation problem: noise applied only to feature channels, positional encoding concatenated after noising as conditioning. This mirrors how <code>GraphEFM</code> separates <code>grid_prev_embedder</code> (prior, no target) from <code>grid_current_embedder</code> (encoder, sees target).
#171 – NNJA-AI Dataset Loader (289 additions)	Dynamic 50TB+ dataset loader with coordinate standardization (<code>LAT</code> → <code>latitude</code> , <code>Lon</code> → <code>longitude</code>), PyTorch <code>DataLoader</code> adapters, 6 unit tests	Relevant to datastore integration. Neural-LAM's datastore refactoring follows the same pattern of abstracting data access behind a clean API.
#181 – Thermalizer Bug Fix	Fixed channel mismatch in diffusion step, score model predicted <code>[B, features, H, W]</code> but denoising operated	Demonstrates deep tensor debugging. The kind of shape-mismatch debugging that will be essential when porting Graph-EFM's <code>(B, S, T, N, d_f)</code> ensemble tensors.

PR	What I Built	Relevance to Project 3
	on [B, features+2, H, W]	

5.2 From-Scratch Weather Transformer

[weather-transformer-scratch](#) – Physics-aware Vision Transformer for weather forecasting:

- Built from scratch on ERA5/WeatherBench2 data
- 74 tests, 27% RMSE improvement over persistence baseline
- Physics-informed loss function
- Full training pipeline with PyTorch Lightning
- Same data domain (ERA5) and framework (Lightning) as Neural-LAM

5.3 Neural-LAM Codebase Engagement

- [PR #189 Code Review](#): Technical comment on zero-std edge case, identified that the same guard should be applied in `datastore/base.py` → `_standardize_datarray()` and suggested `loguru.logger.warning()` for debugging. Received positive feedback from @sadamov.
- **Codebase Deep-Dive**: Performed method-level classification of all 18 `ARModel` methods, full analysis of `GraphEFM` on `prob_model_lam` including all 17 methods, and paper-to-code mapping for Graph-EFM (arXiv:2406.04759) and Diffusion-LAM (arXiv:2502.07532). See **Appendix** for full analysis documents.
- **Local environment setup**: Forked, cloned, ran test suite (79/110 pass – 31 failures are expected remote data access issues from ECMWF object store; 2 are gloo distributed backend issues specific to my local machine).

6. Technical Analysis (Condensed)

6.1 ARModel Method Classification

I classified every method in `ar_model.py` into Joel's proposed #49 categories. Key finding: **training_step, validation_step, and test_step are the most tangled**: each touches [LOSS], [METRICS], and [FORECASTER_MODULE].

The clean separation points are:

- `unroll_prediction` → **ARForecaster**
- Lightning lifecycle → **ForecasterModule**
- Loss and metrics → composable/overridable methods in `ForecasterModule`

6.2 GraphEFM Coupling Points

From reading `graph_efm.py` (1129 lines on `prob_model_lam`), I identified **9 concrete merge conflicts**:

1. `training_step` completely replaced (ELBO vs MSE)
2. GraphEFM **bypasses** `BaseGraphModel` (duplicated graph loading)
3. Batch format: 3-element vs 4-element (missing `batch_times`)
4. Hardcoded `wandb.log()` vs `self.logger`
5. `constants` module vs `datastore` API
6. `border_mask` vs `boundary_mask` naming
7. Ensemble metric storage alongside deterministic metrics
8. `plot_examples` signature mismatch (GraphEFM omits `split` parameter)
9. `on_validation_epoch_end` ordering asymmetry with `on_test_epoch_end`

Each conflict has a clear resolution path in the proposed architecture.

6.3 Paper → Code Mapping

Paper Concept	Code Reference
Prior $p(z x_{<t})$	<code>prior_model(grid_prev_emb, ...)</code> in <code>predict_step</code> L600
Encoder $q(z x_{\leq t})$	<code>encoder(grid_current_emb, ...)</code> in <code>compute_step_loss</code> L403
$ELBO = E_q[\log p(x z)] - \beta \cdot KL$	<code>training_step</code> L533-545
Ensemble generation	<code>sample_trajectories()</code> calls <code>unroll_prediction</code> S times
CRPS (ensemble)	<code>metrics.crps_ens:</code> biased/unbiased/almost-fair estimator modes (PR #226)

7. Communication & Collaboration Plan

- **Weekly sync:** Brief written update in `#gsoc-project3` Slack channel every Monday
- **Short calls:** Weekly or biweekly video/voice calls with mentors for design discussions and progress check-ins.

- **PR cadence:** One focused PR per phase milestone (not drive-by PRs). Each PR includes tests and documentation.
 - **Code review:** All code submitted for mentor review before merging. I welcome early feedback on design directions.
 - **Blockers:** Any blockers communicated within 24 hours via Slack, not left until the weekly update.
-

8. AI Transparency Statement

Per Neural-LAM's policy on AI tooling: I have used AI tools (GitHub Copilot, Gemini) during my pre-proposal preparation for **code architecture analysis, line-by-line codebase classification, and language refinement** of my observation notes. All architectural understanding, design decisions, and technical analysis represent my own work, I read every line of `ar_model.py`, `graph_efm.py`, and the supporting files, and can explain every design choice in this proposal. During the GSoC coding period, I will use AI tools only as a coding assistant, and I take full responsibility for all contributed code.

9. About Me

I'm a **Ceramic Engineering undergraduate at NIT Rourkela** with deep self-directed expertise in scientific ML and weather prediction. While my formal coursework is in materials science, I have built deep hands-on experience in Python, PyTorch, and ML model development through personal projects and open-source contributions, from building neural weather models from scratch (Weather Transformer on ERA5) to contributing production-quality code to Open Climate Fix's `graph_weather` repository (3 merged PRs).

What distinguishes my background for this project is the **overlap between my OCF diffusion model work and the probabilistic forecasting models** this project aims to integrate. The ThermalizerLayer I built uses the same fundamental pattern as Graph-EFM: separating conditioning paths from generative paths, managing dual forward modes (training vs inference), and handling stochastic sampling within an autoregressive framework.

I have no other summer commitments and can dedicate **35 hours/week** throughout the program. My university end-semester exams finish in the 2nd week of April 2026, well before Community Bonding begins.

10. Post-GSoC Plans

I plan to continue contributing to Neural-LAM after GSoC, particularly:

- **Diffusion-LAM integration**, building on the hierarchy established in this project
 - **Global forecasting integration** (Project 4) given my ERA5 experience
 - Reviewing and mentoring future contributors on the refactored architecture
-

References

4. Oskarsson, J., Landelius, T., Deisenroth, M. P., & Lindsten, F. (2024). *Probabilistic Weather Forecasting with Hierarchical Graph Neural Networks*. [arXiv:2406.04759](https://arxiv.org/abs/2406.04759)
 5. Oskarsson, J., Larsson, E., Landelius, T., & Lindsten, F. (2025). Probabilistic Limited Area Weather Forecasting with Diffusion. [arXiv:2502.07532](https://arxiv.org/abs/2502.07532)
 6. Issue #49: Refactor model class hierarchy (<https://github.com/mllam/neural-lam/issues/49>)
 7. Issue #62: Merge Graph-EFM from prob_model_lam(<https://github.com/mllam/neural-lam/issues/62>)
 8. PR #208: Deterministic hierarchy refactoring (draft) (<https://github.com/mllam/neural-lam/pull/208>)
 9. Issue #335: [RFC/Design] Standardize probabilistic vs deterministic return contract (<https://github.com/mllam/neural-lam/issues/335>)
-

Appendix: Detailed Codebase Analysis

The following analysis documents are available in my fork for full review:

- [Codebase Classification](#) - Method-level classification of all 18 `ARModel` methods and all 17 `GraphEFM` methods, mapped to the Issue #49 hierarchy categories
- [Graph-EFM Deep Analysis](#) - Full architectural analysis of Graph-EFM on `prob_model_lam`, including all 9 merge conflicts, coupling points, and integration strategy
- [Paper → Code Mapping](#) - Equation-to-code mapping for Graph-EFM (arXiv:2406.04759) and Diffusion-LAM (arXiv:2502.07532), with line-level references